

# # ALG. Relationnel :

→ Ensemble d'opération qui permettent de manipuler des relation dans une base de données relationnelle.

## • Contraintes d'intégrité :

→ Contraintes d'application.  
→ Contrainte de structure.

- + Contrainte de domaine
- + Contrainte de relation
- + Contrainte de référence

condition sur les valeurs.

une seule clé primaire dans une relation.

Notion de clé étrangère

## • Algèbre relationnelle

### Opérateurs :

### # Opérations de base :

(Lister)

+ Projection : conserver des attributs.

$\Pi_{nom, prenom}$  (Personne)

+ Selection : conserver les tuples qui vérifient une condition.

= , et :  $\wedge$   
≥ , ou :  $\vee$   
< , Non :  $\neg$

$\sigma_{(adresse = 'casa')}$  (Personne)

$\Pi_{nom, prenom}(\sigma_{(sexe = 'F')})$  (Personne)

+ Union : (U) (soit)

$\Pi_{np}(\text{Enseignant}) \cup \Pi_{np}(\text{Etudiant})$

+ Différence :

de 2 relations.

$(R - S) \rightarrow$  les tuples qui se trouvent dans R et ni est pas dans S

$\Pi_{ne}(\text{Inscrit}) - \Pi_{ne}(\text{Obtenu})$   
(qui n'ont pas réussi)

+ Produit cartésien :

l'ensemble de toutes les concaténation possibles de tuples de R et T.  
Symbole :  $\times$

### # Opérateur syntaxique :

+ Renommer :

o syntaxe :

$\rho_{[nom\_attribut : nouveau\_nom, \dots]}(R)$

### # Opérations déduites :

+ Jointure naturelle : (équijointure)

concaténation de 2 relations ayant des attributs communs.

Symbole :  $\bowtie$

$R_1 \bowtie_E R_2 \Leftrightarrow \sigma_E(R_1 \times R_2)$

5



Personne  $\bowtie$  Etudiant

Personne.np = Etudiant.np  
(même valeur)

+ **Thêta - Jointure :**

concaténation de 2 relations, et l'utilisation d'une condition quelconque sur les attributs des 2 relations.

$\Pi_{NE1, NE2}$  (Etudiant 1  $\bowtie$  Etudiant 2)  
date N1 = date N2

(condition de nées  
même jour).

# **Intersection :** (n) (Ans)

$\Pi_{NE}$  (Inscrit)  $\cap$   $\Pi_{NE}$  (obtenus)

(Inscrit et réussie)

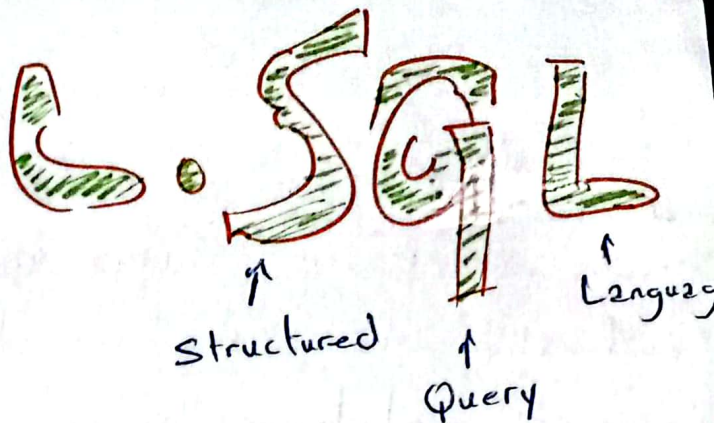
+ **Division :**

symbole :  $\div$

$R(A_1, A_2, A_3, B_1, B_2)$

et  $S(B_1, B_2)$

$R \div S = T(A_1, A_2, A_3)$



→ Language de ~~données~~ requête structurée, sert à gérer les bases de données relationnelles.

• **les catégories de L. SQL :**  
+ **LDD** (Language de définition des données).

+ **LMD** (Language de manipulation de données).

+ **LCD** (Language de contrôle de données).

+ **LT** (Language de contrôle de transaction).

• **LDD :**

→ Créer : Create

→ Modifier : Alter

→ Supprimer : Drop

• **LMD :**

→ Insérer : Insert

→ Mettre à jour : Update

→ Supprimer : delete

→ Interroger : Select.

## • LCD:

- Donner un droit: Grant
- Retirer un droit: Revoke

## • LT:

- Annuler une transaction: Rollback.
- Valider une transaction: Commit.

## # les opérations du LDD:

### • Creation d'une BDD:

```
create database nom
```

- Si existe erreur, on fait cela:

```
create database if not exists nom
```

### • Suppression d'une BDD:

```
drop database nom
```

- Si n'existe pas erreur, on fait cela:

```
drop database if exists nom
```

### • Creation d'une table:

exemple:

```
create table user  
(id int,  
nom varchar(100),  
email varchar(100),  
date_naissance date);
```

### • Définir des contraintes pour chaque colonne:

- Not Null:

exemple:

```
create table clients  
(id int primary key,  
nom varchar(50) not null);
```

- default:

```
create table clients  
(ville varchar(255) not null  
default 'cas2');
```

- primary Key:

```
create table user  
(id int primary key);
```

- les clés secondaires (unique):  
(Autorise valeur null).

```
create table clients  
(email varchar(100),  
unique(email));
```

- les clés étrangères:

- foreign Key → fais référence à une ligne d'une autre table.

```
create table commandes  
(client_id INT,  
foreign key (client_id)  
references clients(id_client));
```



→ la contrainte check:  
• définir une condition.

```
create table emp(  
genre char check(  
genre in ('f', 'm', 'F', 'M'))
```

• Suppression d'une table

```
drop table nom
```

• Modification d'une table.

```
alter table nom Action  
description.
```

\* Action :

- Add
- modify
- drop
- rename

```
alter table produit
```

```
modify column cat varchar(50);
```

• Ajout et retrait d'une contrainte

```
alter table client
```

```
add unique (nom, adresse)
```

```
alter table client
```

```
add foreign key (cat)  
reference categorie;
```

```
alter table client
```

```
modify cat not null;
```

• Les opérations du LMD:

• Insertion de données:  
→ ajouter nouvelles ligne.

```
insert into nom_table
```

```
(champ1, ...)
```

```
values (val1, ...);
```

• Insertion de plusieurs lignes  
à la fois:

```
insert into clients
```

```
values (234, 'Pary', '51100'),  
(235, 'Cas2', '52200');
```

→ si valeur pas connu, met null.

• Float: ex: 3.14

• Char et varchar: 'Soukaina'

• date: '2222-mm-jj'

• time: 'hh:mm:ss'

• Mise à jour des données:

• modifier valeur des champs  
dans une table.

```
update nom_table
```

```
set champ1 = valeur1
```

```
where condition;
```

update client

set adresse Clt = '46, rue F'

where No clt = 203;

## • Suppression de données :

→ supprimer des enreg. pas  
la structure de table.

selon critères de comm. where.

→ tous les enreg. va supprimer  
avec l'absence de where.

delete from client

where Nom client = 'Soukainz';

## • Language d'interrogation des bases de données (LID).

### • Projection :

sélection d'attributs  
dans table.

select [All | distinct]

from nom de relation

→ N.B : My SQL evalue  
la clause from avant  
la clause select.

select nom p, categorie  
from produit.

### • Selection :

• récupérer des données d'une ou plusieurs  
tables en fonction des critères spécifiques.  
• on utilise la commande select et la clause  
where.

select \* ← pour lister  
tous les colonnes.

from nom\_table

where (condition de recherche)  
← categorie = 'informatique'

## % Alias ou Synonyme de relation :

• est un nom temporaire, facile écriture  
des requête SQL.

• on utilise dans :   
select : renommer colonnes  
from : met nom tabl  
court.

(1) select nom, prix \* 1.1  
As nouveau\_prix.

(2) select p.nom p, p.prix  
from produits As p

## % 3 formes pour désigner les attributs d'une relation :

nom\_attribut (nom ambigu)  
nom\_relation . nom\_attribut.  
Alias\_relation . nom\_attribut.

## % La commande where :

→ Sélectionner des lignes à une condition  
logique.

select nom\_champ  
from Nom\_table  
where condition

## → les opérateurs à utiliser :

|                                                    |                                                       |
|----------------------------------------------------|-------------------------------------------------------|
| opérateur logiques:<br>AND - OR - NOT              | opérateurs arithmétique.<br>+ /<br>- %<br>*           |
| comparateur<br>Logiques :<br>IN<br>LIKE<br>between | comparateur<br>arithmétique :<br>= ><br>!= <=<br>< >= |



## → Opérateurs Logique AND, OR

```
select Nom_champ
from Table
where condition 1 AND
      (condition 2 OR condition 3)
```

## → l'opérateur IN:

- vérifie si une colonne égale à une valeur ou autre.

```
select Nom_champ
from Nom_table
where Nom_champ IN (val 1, val 2);
```

Not in recherche les lignes qui ne sont pas égales à ces valeurs.

## → l'opérateur BETWEEN:

- retourne les lignes qui appartiennent à l'intervalle.

```
select *
from Nom_table
where Nom_colonne BETWEEN
      'valeur 1' AND 'valeur 2'
```

→ not between (contraires)

## → l'opération like:

```
select *
from Nom_table
where Nom_colonne like modèle
```

✓ like 'a%'   
 ✓ like 'a%'   
 ✓ like '%a%'   
 ✓ like 'pa%.on'   
 ✓ like 'a\_c'

remplace tous les autres caractères.  
 "underscore" remplace caractère quelconque.

## → l'opérateur is null / is not null:

- retourner les enregistrements qui sont nuls ou pas nuls.

```
select *
from nom_table
where nom_colonne is null;
```

## % clause order by: (le tri).

```
select colonne 1, colonne 2
from table
order by colonne 1 desc,
      colonne 2 asc;
```

- NB:
- par défaut croissant.
  - les val nuls sont tjrs en tête.

## • Fonction intégrées: (P. 50)

- manipuler données utilisateurs ou données du système.

## → Fonctions mathématiques.

→ renvoient des valeurs numériques.

- abs()
- ceil()
- floor()
- mod()
- round()
- truncate()

\* pour augmenter:

$$E' = A + \frac{E}{100}$$

\* pour diminuer

$$E' = A - \frac{E}{100}$$

```
select nomemp, salaire,
      round(salaire * 1.15) as
      'New salary'
```

```
from employe;
```

## → Fonctions de traitement de chaînes:

- manipuler les données textuelles:

- concat
- instr
- length
- left
- lower
- trim
- ltrim
- rtrim
- replace
- right
- substring
- substring\_index
- lpad
- find\_in\_set
- format
- upper

→ Fonctions de manipulation de dates : manipuler des données de types date

- curdate
- datediff
- day
- date\_add
- date\_sub
- month
- str\_to\_date
- sysdate
- week
- year
- date\_format
- dayname
- dayofweek
- extract
- last\_day
- now
- timediff
- timestampdiff
- weekday

→ Fonction de conversion :  
→ transformer type valeur à un autre.

◦ date\_format (date, 'format')

\* %Y \* %d \* %h  
\* %y \* %W \* %i  
\* %M \* %H \* %S  
\* %am

```
select codemp, nomemp,
       date_format(date recrut,
                    '%Y-%m') as annee_mois
from employe;
```

◦ str\_to\_date ('chaîne', 'format')

◦ cast() AND convert()  
→ convertir en différent types.

\* Conversion en texte.

cast (expression AS char)  
ou

convert (expression, char)

\* Conversion en Nbre :

cast (expression AS decimal)

ou  
convert (expression, decimal)

1. Conversion et gestion des valeurs NULL :

◦ En SQL, null représente une valeur inconnue.

◦ Les opérateurs sur valeurs Null renvoient null.

ifnull (expression, valeur\_de\_replacement)

→ remplacer une valeur null par une valeur.

coalesce (exp 1, exp 2, ...)

→ retourne la première valeur non null.



# Fonctions Intégrées

## A) Fonctions

### Mathématiques :

- **Abs** (-5)  $\sim$  5 ← valeur absolue
- **ceil** (4.2)  $\sim$  5 ← retourne la valeur grande
- **ceil** (-4.8)  $\sim$  -4
- **floor** (4.8)  $\sim$  4
- **floor** (-4.2)  $\sim$  -5 ← valeur petite
- **mod** (17.5)  $\sim$  2 ← reste
- **Round** (3.1415, 2)  $\sim$  3.14
- **Round** (3.14712)  $\sim$  3.15
- **Round** (3.147)  $\sim$  3
- **truncate** (3.147, 2)  $\sim$  3.14
- **truncate** (3.147, 0)  $\sim$  3

## B) Fonction de traitement de chaînes :

- **concat** ('une', 'maison')  $\rightarrow$  unemaison
- **instr** ('database', 'base')  $\rightarrow$  5 (position du mot base)
- **Length** ('mysql')  $\rightarrow$  5 (nombre de caractère)
- **upper** (souka)  $\rightarrow$  SOUKA
- **lower** (MYSQL)  $\rightarrow$  mysql

- **trim** ('H' from 'Helloworld')  $\rightarrow$  elloword suppression seulement au début et à la fin
- **ltrim** (' Bonjour')  $\rightarrow$  Bonjour sans espace à l'au début
- **Rtrim** (' Bonjour ')  $\rightarrow$  Bonjour sans espace à la fin
- **left** (' Bonjour', 3)  $\rightarrow$  Bon ← extraire nbre caractères à partir de gauche
- **replace** ('jack', 'j', 'B')  $\rightarrow$  Black start: 2ème espace
- **right** (' Bonjour', 3)  $\rightarrow$  our
- **substring** (' Bonjour', 2, 3)  $\rightarrow$  ong ↑ longueur
- **substring** (' Bonjour', 4)  $\rightarrow$  jour ↑ start
- **substring-index** (string, delimiter, count) separateurs { ' ' } si positif (avant) si négatif (après)
- **substring-index** ('a,b,c,d', 'b', 2)  $\rightarrow$  'a,b'
- **substring-index** ('a,b,c,d', 'b', -2)  $\rightarrow$  'c,d'
- **lpad** ('123', 6, ' \$')  $\rightarrow$  \$ \$ \$ 123 ajouter à gauche
- **find\_in\_set** (valeur, liste) renvoie la position de la valeur
- **find\_in\_set** ('b', 'a,b,c,d')  $\rightarrow$  2
- **format** (nbre, decimales [, locale]) combien des chiffres après la virgule   
fr-FR en-US liste des valeurs séparées par des virgules
- **format** (12345.6789, 2)  $\rightarrow$  12,345.68 en-US par défaut
- **format** (12345.6789, 2, 'fr-FR')  $\rightarrow$  12 345,68



# ! Fonctions de manipulation de dates :

- **curdate()** → 2025-05-10   
 ← date actuelle
- **datediff** ('2025-05-10', '2025-05-01') → 9   
 ← différence de 2 dates
- **day** ('2025-05-10') → 10   
 ← extraire le jour de mois
- **date\_add** (date, interval quantity unit)   
 ← extraire le jour de mois
- **date\_add** ('2025-05-10', interval 5 day) → 2025-05-15
- **date\_sub** ('2025-05-10', interval 5 day) → 2025-05-05
- **date\_format** → une fonction de conversion.
- **dayname** ('2025-05-10') → samedi   
 ← obtenir le nom de jour.
- **dayofweek** ('2025-05-10') → 7 (samedi)   
 ← à partir de 1 qui représente dimanche.
- **extract** (year from '2025-05-10') → 2025   
 ← récupérer le numéro de jour.
- **last\_day** ('2025-05-10') → obtenir le dernier jour du mois
- **now()** → 2025-05-10 14:30:00   
 ← la date et l'heure actuelle.
- **month** ('2025-05-10') → 5   
 ← extraire le mois.
- **str\_to\_date** → une fonction de conversion.
- **sysdate()** → renvoie la date actuelle
- **timediff** ('14:30:00', '12:30:00') → 02:30:00
- **timestampdiff** (month, '2025-01-01', '2025-05-10') → 4   
 ← différence en mois.
- **week** ('2025-05-10') → 19   
 ← renvoie le numéro de semaines dans l'année.

- **weekday** ('2025-05-10') → 5   
 ← 0 → lundi, 5 → samedi   
 ← obtenir le jour de la semaine sous forme numérique.
- **year** ('2025-05-10') → 2025   
 ← extraire l'année.

## ! Fonction de conversion:

Format:

- %Y
- %y
- %M
- %m
- %d
- %W
- %H
- %h
- %i
- %s

Année sur 4 chiffres  
Année sur 2 chiffres  
Mois en texte  
mois en chiffres  
Jour de mois  
jour de la semaine  
Heure (24h)  
Heure (12h)  
minute  
seconde

2024  
24  
March  
03  
23  
saturday  
15  
03  
45  
30

- **date\_format** ('2025-05-10', '%d/%m/%Y') → 10/05/2025   
 ← afficher une date dans un format personnalisé
- **str\_to\_date** ('10-05-2025 14:30:00', '%d-%m-%Y %H:%i:%s') → 2025-05-10 14:30:00   
 ← convertir chaîne de caractère en une date.

- **cast()** **convert()**



- exprimer le produit cartésien

```
select *
from produit, stock;
```

→ Produit cartésien (norme SQL2)

```
select nomemp, nomdep
from employe cross join
      departement;
```

- retourne tous les combinaisons possible de 2 tables.

- Exprimer la jointure?

```
select nomemp, nomdep
from employe, departement
where employe.coddep =
      departement.coddep
```

↑  
Jointure  
ancienne  
notation.

```
select d.nomdep
from departement d, employe e
where d.coddep = e.coddep
and e.nomemp = 'Ali';
```

% Jointure (norme SQL2):

```
select nomemp, nomdep
from employe join departement
      on employe.coddep =
      departement.coddep;
```

% Jointure naturelle:

- combine autom. deux tables en se basant sur les colonnes communes.

```
select nomemp, nomdep, codedep
from employe naturel join
      departement
```

- Par défaut naturelle join joint tous les colonnes.
- si tu veux spécifier les colonnes:

```
select nomemp, nomdep
from employe join departement
      using (codedep, ...);
```

% Left Join:

- récupère les lignes de la table de gauche et les associe avec celles de la table droite.
- si pas de correspondance, les valeurs de table droite affiche null.

```
select etudiant.nom, Note, note
from etudiant left join Note on
      etudiant.id = Note.etudiant_id
```

% Right Join:

- le contraire de Left Join.

```
select Etudiant.nom, Notes, note
from Etudiants Right join Notes
      on Etudiants.id = Notes.etudiant_id
```

N.B: garder tous les notes, même si l'étudiant n'existe pas.



## % Self join: (Auto jointure)

- jointure d'une table avec elle-même
- pour comparer les lignes d'une table entre elles.
- on utilise des alias pour éviter la confusion.

```
select A.colonne 1 B.colonne 2
```

```
on A.colonne commune =  
B.colonne commune;
```

```
select E.nom as Employé,  
       S.M.nom as Manager  
from   Employes E  
       join Employes M on  
       E.manager.id = M.id;
```

## % jointure non-équijointure:

- Rappel des équijointures:

```
from employe e  
join departement d on  
e.coddep = d.coddep
```

→ si on a d'autre chose (>, <, between, and ...)  
on parle alors sur non-équijointure (théorème jointure on Alg. Rel.)

```
select e.nomemp, d.nomdep,  
       e.salaire, s.grade
```

```
from employe e  
join salgrade s on  
e.salaire between s.salmin  
and s.salmax;
```

## Les fonctions de groupe:

- calcule la moyenne
  - Avg (salaire) ou Avg (all salaire)
  - Avg (distinct salaire) → on compte la valeur répéter une seule fois.
- renvoie plus petite valeur.
  - min (salaire)

- renvoie la plus grande valeur
  - max (salaire)

- somme des valeurs
  - sum (salaire)
  - sum (distinct salaire)

- compte tous les lignes (y compris Null).
  - count (\*)

- count (salaire) → ignore le null
- count (distinct salaire)

- Mesure la dispersion des valeurs par rapport à la moyenne.

- variance (all salaire)
- variance (distinct salaire)

- Ecart-type: à quel point les valeurs sont éloignées de la moyenne.

- stddev (all salaire)
- stddev (distinct salaire)

```
select max (salaire) as salaire_max,  
       min (salaire) as salaire_min,  
       Avg (salaire) as salaire_moyen  
from employe;
```

N.B. les fonctions de groupe peuvent apparaître dans 'select' ou 'having'.

- tous les fct de group à l'exception de count(\*) ignorent les valeurs null.  
solution: coalesce()
- ```
select Avg (coalesce (prime, 0)) as moyenne_prime  
from employe;
```



- max et min s'utilisant pour tous les types de données.
- Les autres fct n'acceptent que les données numériques.

### % La clause group by :

- rassembler les lignes qui ont la même valeurs dans une colonne.
- peut compter combien il y en a, ou faire des calculs (sum, avg...)

tu peux le met ou non.

```
select coddep, count(*)
from employe
group by coddep.
```

```
select coddep, count(*)
from employe
where salaire > 4000
group by coddep;
```

### % La clause having :

- having filtre après groupe by. alors il filtre les résultats après regroupement.

```
select coddep, sum(salaire) as
total_salaire
from employe
group by coddep
having sum(salaire) > 10000;
```

### • opérations ensemblistes :

- permettent de combiner les résultats de plusieurs requêtes select en une seule.

- Union.
- Union All.
- intersect
- except (or minus in oracle)

- Les règles :

- ⊕ même nbre de colonne dans chaque select
- ⊕ type données compatible (txt txt) (nbr nbr)
- ⊕ Les doublons auto. supprimés sauf si on a utilisé (union all)
- ⊕ Les noms de colonnes de premier select ceux sont qui vont être affichés.
- ⊕ on trouve qu'une seule 'order by' on la met après la dernière select.

```
select id, nom, note from Etudiant1
union
select identifiant, prenom, note from
Etudiant2
```

← même type  
← même nbre de colonne.

- pour trier la résultat :

```
select ... from
union
select ... from
order by note desc;
```

- Les opérateurs :

### % Union :

- \* combine résultat de 2 requêtes sans doublons.

- ⚠ colonnes avec même nbre, types compatibles.

```
select codemp from Employe
union
select coderespon from departement
```

sans doublons → Union

(Union all) → avec les doublons.



## % intersect: (N)

- Affiche les valeurs commune entre deux requêtes.

```
select codemp from employe  
intersect
```

```
select codresp from departement;
```

## % Except: (minus in oracle) (-)

- Affiche les résultats de la 1<sup>er</sup> requête, sauf ceux qui sont aussi dans la seconde.

```
select codemp from Employe
```

```
Except
```

```
select coderespon from dep;
```

## • Sous-interrogation: (sous-requête)

- est une requête imbriquée à l'intérieur d'une autre requête.

```
select nom  
from Employés  
where salaire < (select salaire from  
Employés where  
nom = 'Ali');
```

← requête principale: comparer ce salaire à d'autres employés.

↑ sous-requête: trouver salaire de Ali d'abord.

## % où peut-on utiliser une sous-requête?

- \* Dans select: condition (ligne principal avec seule ligne de sous requête).

```
select nom, (select Avg(salaire)  
from employes) As salaire_moy
```

```
from employes;
```

- \* Dans from: on utilise des alias.

```
select e.nom, e.salaire  
from (select * from employes  
where salaire > 3000)  
As e;
```

- \* Dans where: très courant utilisée.

```
select nom, salaire  
from employes  
where salaire > (select  
Avg(salaire)  
from employes)
```

- \* Dans having: courant de l'utilisé.

```
select departement, Avg(salaire)  
from employes  
group by departement  
having Avg(salaire) > (select  
Avg(salaire)  
from employes)
```

## % Syntaxe:

```
select select_list  
from table  
where expr operator (select  
select_list  
from table);
```

- N.B: → sous requête exécutée une fois avant requête principale

→ opérateurs de comparaison:

→ opérateurs monoligne: >, =, <, >=, <=, <>.

→ opérateurs multi ligne: in, any, all.



## % Principe d'utilisation :

- + tjr il faut mettre on (1)
- + tjr à droite de comparateur.
- + Pas de Order by (Pas de tris dans une sous-requête).
- + on utilise des opér. simple :  $=, >, <, >=, <=, <>$ , si sous-requête donne une seule valeur (mono ligne).
- + on utilise les opér. multiligne : in, any, all, exists — si le contraire.

## % Types de sous-interrogation :

### + mono ligne :

Ne ramènent qu'une seule ligne, utilisant des opérateurs de compr. mono-ligne, qui sont :  $=, >, <, >=, <=, <>$ .

### Exemples :

1) Quels sont les employés ayant le même département que l'employé de code 105 et ayant un salaire inférieur à celui de Ali.

```
select nomemp.  
from employe  
where coddep = (select coddep  
from employe  
where coddep = 105)
```

```
And salaire < (select salaire  
from employe  
where nomemp = 'Ali')
```

2) Afficher les départ. dont le salaire minimum est supérieur au salaire mini. du département D20.

```
select coddep, min(salaire)  
from employe  
groupe by coddep  
having min(salaire) > (select  
min(salaire) from  
employe where  
coddep = 'D20')
```

## Remarque :

→ Logique d'exécution en SQL.

- ⊕ from
- ⊕ where
- ⊕ groupe by
- ⊕ having
- ⊕ select
- ⊕ order by.

## \* Multi-Lignes :

1) Rappel : sous-interrogation et une requête à l'intérieur d'une autre requête.

→ Pourquoi multi-ligne ? car elle peut ramener plusieurs résultat.

→ Quand on peut l'utiliser ?

quand on veut comparer une valeur à un ensemble des valeurs.

(exp: vérifier si une note dans une liste des notes)

→ Les opérateurs utilisés :

• **IN :** sert à dire "est-ce que cette note est dans la liste."

```
where note IN (select note from  
examen)
```

• **NOT IN :** Le contraire.

• **ANY :** "est-ce que la valeur est vraie au moins un des résultat."

(exp: le prix le plus petit que au moins un des prix dans la liste)

```
where prix < ANY (select  
prix from produits)
```

• **ALL :** "est-ce que la valeur est vraie pour tous les résultat"

(exp: le prix le plus petit que tous les prix de la liste).

```
where prix < ALL (select  
prix from produits)
```

$=, >, <$

$>=, !=, <=$



## Remarque:

→ opérateur **IN** est équivalent à **= ANY**

→ opérateur **NOT IN** est équivalent à **!= ALL**

→ **< ANY** : inférieur au maximum.

→ **> ANY** : supérieur au minimum.

→ **> ALL** : supérieur au max

→ **< ALL** : inférieur au min.

## exemples:

(1). Quels sont les employés ayant le salaire le plus bas dans chaque département.

```
select nom emp, salaire, coddep
from employe
where salaire IN (select coddep,
                  min(salaire)
                  from employe
                  group by coddep);
```

↑  
après chercher  
les employés  
ont un de ces  
salaire min

↑  
trouve d'abord  
tous les salaires  
min par dépt.

(2). Afficher les employés dont le salaire est sup. à tous les salaires des employés ayant le poste 'P001', à l'exclusion de 'P001'

```
select codemp, nomemp, salaire, codposte
from employe
where salaire > ALL (select salaire
                    from employe
                    where codpost = 'P001')
And codposte <> 'P001';
```

(3). liste des employés du départ. 10 ayant le même poste que quelqu'un du départ. ventes.

```
select nom emp, coddep
from employe
where coddep = 10 And
codposte = ANY (select
codposte from employe where
coddep = (select coddep
from departement where
nom_dep = 'ventes'));
```

## \* Multi colonnes :

• c'est une sous-requête qui retourne plusieurs colonnes. Elle permet de comparer plusieurs colonnes ensemble. Comme ((salaire, coddep)).

→ syntaxe :

```
select (column, column, ...) IN
(select column, column, ...
from table
where condition);
```

on utilise seulement **IN** **NOT IN**  
Les autres (**ANY**, **ALL**) Non.

```
select column 1, column 2 ...
from table
where (column, column, ...) IN
(select column, column ...
from table
where condition);
```

## exemples:

(1) afficher le nom, le numéro de départ et le salaire de tout employé dont le numéro de département et le salaire sont tous les deux équivalents au numéro de département et au salaire de n'importe quel employé touchant une prime.

```

select nomemp, coddep, salaire
from employe
where (salaire, coddep) IN
      (select e.salaire,
             e.coddep
      from employe e
      where e.prima is not
            null);

```

(2) Afficher le nom de l'employé, le nom de département et le salaire de tous employés dont le salaire et la prime sont tous les deux équivalents au salaire et à la prime de n'importe quel employé du département RH.

```

select e.nomemp, d.nomdep, e.salaire
from employe e join departement d
on e.coddep = d.coddep
where (e.salaire, coalesce(e.prima, 0))
IN (select (e1.salaire, coalesce(e1.prima, 0))
    from employe e1
    join departement d1
    on e1.coddep = d1.coddep
    where d1.nomdep = 'RH');

```

## \* Synchronisées :

### % définition :

Une sous requête synchronisée ou sous requête corrélée est une sous requête qui dépend d'une valeur de la requête principale.

→ il exécute une fois pour chaque ligne de la requête principale.

### % utilisation :

Quand tu fais une req. du type  
 select ... from ...  
 where column 1 = (select ...)  
 et que la valeur cherchée dépend de chaque ligne alors tu ne peux pas utiliser une simple sous requête tu dois utiliser une sous requête synchronisée.

### % Caractéristique :

- ⊕ Elle s'exécute pour chaque ligne de la requête principale.
- ⊕ Elle utilise une colonne de la requête principale dans sa clause

- ⊕ il faut utiliser des alias pour bien distinguer les colonnes si les deux requêtes utilisent la même table.

### exemples :



- trouvez les employés dont le salaire sup. à la moyenne de leur de leur depart.

```
select e.nomemp, e.salaire,
       e.coddep
```

```
from employe e
```

```
where e.salaire > (select Avg(e1.salaire)
```

```
from employe e1
```

```
where e1.coddep
```

```
= e.coddep);
```

Alors ici :

→ e.coddep appartient à la requête principale.

→ La sous requête utilise celle valeur pour calculer la moyenne pour chaque département.

Donc cette sous req. est synchronisée

⚠ Une sous interrogation synchronisée peut ramener une ou plusieurs lignes différents opérateurs peuvent être employés (=, >, <, >=, Exists).

%, Syntaxe avec Exists :

```
select *
```

```
from table alias 1
```

```
where [Not] exists (select *
```

```
from table alias 2
```

```
where alias2.idE =
```

```
alias1.idE);
```

%, Exemple :

- Afficher tous les employés qui ne sont pas responsables d'autres employés.

```
select e1.nomemp
```

```
from employe e1
```

```
where Not exists (select
```

```
e2.codemp from employe e2
```

```
where e2.superieur =
```

```
e1.codemp);
```